FOUNDATIONS

AI · ML · DS / REVISION NOTES

PYTHON DEEP DIVE

Python Decorators

Functions as objects, closures, and writing your own decorators

1
FIGURES

6
PAGES

Compiled October 2026
github.com/ankitgupta1729/AI-ML-DS

1 · Suppose, I have written the code as

```
def timer_dec(base_fn):
    # Code to decorate
    return enhanced_fn

@timer_dec
def brew_tea():
    # Code to brew
```

Here, @timer_dec is a decorator syntax. timer_dec is a decorator. A decorator like timer_dec is itself a function. The purpose of this function is to decorate or enhance a base function "base_fn" that passed as an argument and return an enhanced function.

By writing @timer_dec on the top of the definition of brew_tea(), we tell python that the base function "brew_tea()" here must be enhanced by the decorator "timer_dec" before it is used.

After receiving base function "base_fn" as an input, the decorator bundles it with additional features without modifying the base function's original code. Once these new features are added, decorator returns the enhanced version of the function. This enhanced version of the function is what python will use when brew_tea() is called. This is how decorators work.

2. Why use decorators to add on extra code when we could simply include the additional operations in the original function definition ?

It means we can also write extra code to add extra features for a function like:

```
def brew_tea():
    # Code
    # Code to brew
    # Code
```

Now, suppose, initially we have:

```
import time
def brew_tea():
    print("Brewing Tea...")
    time.sleep(1)
    print("Tea is ready!")
brew_tea() # function is created for this purpose and it will take approx 1 sec
```

Now, if we add additional features in this function as:

```
def brew_tea():
    start_time = time.time()
    print("Brewing Tea...")
    time.sleep(1)
    print("Tea is ready!")
    end_time = time.time()
    print(f"Task time: {end_time-start_time} seconds")
brew_tea() # function with additional feature to measure exact time
```

There are issues with this approach. The brew_tea() function violates the single responsibility principle by performing 2 different tasks i.e. brewing tea and timing the process. In programming, functions should focus on a single well-defined responsibility to make code reusable.

Suppose, we also have the matcha making function:

```
def make_matcha():
    start_time = time.time()
    print("Making Macha...")
    time.sleep(1)
    print("Macha is ready!")
    end_time = time.time()
    print(f"Task time: {end_time-start_time} seconds")
make_matcha()
```

But duplicating code is not ideal since it make codebase repetitive and harder to maintain. Decorators offer a great solution to these problems.

3. For simplicity, first we decorate the brew_tea() function and we will bring make_match() later. Since, decorators are just functions, to create one, we use the `def` keyword. In this example, the purpose of decorator is to time the execution of a function. Remember that decorators take base function as an input.

To apply decorators in python, we have 2 options:

- We can call the decorator function and pass the function which we want to decorate as an argument.

```
def timer_dec(base_fun):
    def enhanced_fun():
        start_time = time.time()
        base_fun()
        end_time = time.time()
        print(f"Task time: {end_time-start_time} seconds")
    return enhanced_fun

def brew_tea():
    print("Brewing Tea...")
    time.sleep(1)
    print("Tea is ready!")
timer_dec(brew_tea)
```

Running this code, we get a function object. This function object is the enhanced function that the timer_dec returned.

To call this function later in this program, we can give it a name by assign it to a variable and then call it as:

```
def timer_dec(base_fun):
    def enhanced_fun():
        start_time = time.time()
        base_fun()
        end_time = time.time()
        print(f"Task time: {end_time-start_time} seconds")
    return enhanced_fun

def brew_tea():
```

```
    print("Brewing Tea...")
    time.sleep(1)
    print("Tea is ready!")
dec_brew_tea = timer_dec(brew_tea)
dec_brew_tea()
```

By using a decorator, we have added features to brew_tea() without modifying its original code.

- Using @ syntax:

```
def timer_dec(base_fun):
    def enhanced_fun():
        start_time = time.time()
        base_fun()
        end_time = time.time()
        print(f"Task time: {end_time-start_time} seconds")
    return enhanced_fun
@timer_dec
def brew_tea():
    print("Brewing Tea...")
    time.sleep(1)
    print("Tea is ready!")
brew_tea()
```

Writing this is equivalent to applying the decorator manually. Here running brew_tea() runs both the original function and the decorator function.

Decorator function also help us to reuse code.

4. We can use decorators for multiple functions as well.

```
def timer_dec(base_fun):
    def enhanced_fun():
        start_time = time.time()
        base_fun()
        end_time = time.time()
        print(f"Task time: {end_time-start_time} seconds")
    return enhanced_fun
@timer_dec
def brew_tea():
    print("Brewing Tea...")
    time.sleep(1)
    print("Tea is ready!")
@timer_dec
def make_matcha():
    print("Making Matcha...")
    time.sleep(2)
    print("Matcha is ready!")
brew_tea()
make_matcha()
```

5. To decorate functions that do have parameters, we need to do a bit more work.

```
def timer_dec(base_fun):
    def enhanced_fun():
        start_time = time.time()
        base_fun()
        end_time = time.time()
        print(f"Task time: {end_time-start_time} seconds")
    return enhanced_fun
@timer_dec
```

```
def brew_tea(tea_type,steep_time):
    print("Brewing {tea_type} Tea...")
    time.sleep(steep_time)
    print("Tea is ready!")
@timer_dec
def make_matcha():
    print("Making Matcha...")
    time.sleep(2)
    print("Matcha is ready!")
brew_tea("green",1)
make_matcha()
```

It gives the error as:

```
TypeError: timer_dec.<locals>.enhanced_fun() takes 0 positional arguments but 2 were given
```

Remember, when we decorate brew_tea with the timing decorator, calling `brew_tea("green",1)` actually triggers a call to the enhanced_fun() function, which does not expect any arguments.

We can fix it for brew_tea() function as:

```
def timer_dec(base_fun):
    def enhanced_fun(tea_type,steep_time):
        start_time = time.time()
        base_fun(tea_type,steep_time)
        end_time = time.time()
        print(f"Task time: {end_time-start_time} seconds")
    return enhanced_fun
@timer_dec
def brew_tea(tea_type,steep_time):
    print("Brewing {tea_type} Tea...")
    time.sleep(steep_time)
    print("Tea is ready!")
@timer_dec
def make_matcha():
    print("Making Matcha...")
    time.sleep(2)
    print("Matcha is ready!")
brew_tea("green",1)
make_matcha()
```

Now, getting the error for make_matcha() function as:

```
TypeError: timer_dec.<locals>.enhanced_fun() missing 2 required positional arguments:
'tea_type' and 'steep_time'
```

How to make it flexible for both the functions ?

```
import time
from datetime import datetime, timedelta

def timer_dec(base_fun):
    def enhanced_fun(*args,**kwargs):
        start_time = time.time()
        result = base_fun(*args,**kwargs)
        end_time = time.time()
        print(f"Task time: {end_time-start_time} seconds")
        return result
    return enhanced_fun
@timer_dec
```

```
def brew_tea(tea_type,steep_time):
    print("Brewing {tea_type} Tea...")
    time.sleep(steep_time)
    print("Tea is ready!")
@timer_dec
def make_matcha():
    print("Making Matcha...")
    time.sleep(2)
    print("Matcha is ready!")
    return f"Drink matcha by {datetime.now()+timedelta(minutes=30)}"
brew_tea(tea_type="green",steep_time=1)
print(make_matcha())
```

Now, both the functions work fine.